

Identify Spring Boot, Tomcat, WildFly, WebLogic, and WebSphere applications from logs and config

You can identify **Spring Boot**, **Tomcat**, **WildFly**, **WebLogic**, and **WebSphere** applications by looking for specific keywords, class names, directory structures, and file extensions in their configuration files and deployment logs.

Here is a direct comparison to help you quickly signature-match these Java technologies:

Technology	Key Config Files	Distinctive Log Keywords / Packages	Default Ports
Spring Boot	<u>application.properties</u> , <u>application.yml</u>	<u>SpringApplication</u> , <u>org.springframework.boot</u>	<u>8080</u>
Tomcat	<u>server.xml</u> , <u>web.xml</u> , <u>context.xml</u>	<u>org.apache.catalina</u> , <u>Coyote</u>	<u>8080</u>
WildFly	<u>standalone.xml</u> , <u>domain.xml</u>	<u>org.jboss.as</u> , <u>WFLYSRV</u> , <u>Undertow</u>	<u>8080</u> , <u>9990</u>
WebLogic	<u>config.xml</u> , <u>weblogic.xml</u>	<u><BEA-000000></u> , <u>weblogic.Server</u>	<u>7001</u>
WebSphere	<u>server.xml</u> (Liberty), <u>virtualhosts.xml</u>	<u>[WAS]</u> , <u>com.ibm.ws</u> , <u>CWPKI</u>	<u>9080</u> , <u>9443</u>

Log Signatures

Spring Boot

- **ASCII Art:** By default, logs start with a large ASCII art banner reading :: Spring Boot ::.
- **Initialization:** Look for Starting Application using Java... and Started Application in X seconds.
- **Packages:** Prominent log lines containing org.springframework.boot or org.springframework.web.

Apache Tomcat

- **Server Info:** Look for Starting service [Catalina] and Starting Servlet engine: [Apache Tomcat/X.Y.Z].
- **Connectors:** Look for Initializing Coyote HTTP/1.1 on http-nio-8080.

WildFly (formerly JBoss)

- **Server Boot:** Logs will explicitly print WFLYSRV0025: WildFly Full X.Y.Z "Flavor" started.

- **Subsystems:** Look for logs starting with codes like WFLY, JBILD, or references to the Undertow web subsystem.

Oracle WebLogic

- **Log Format:** WebLogic logs use a highly specific format enclosed in angle brackets: ####<Mar 10, 2026 ...> <Info> <WebLogicServer> ... <BEA-000365>.
- **State Changes:** Look for Server state changed to RUNNING.

IBM WebSphere (Traditional & Liberty)

- **Traditional WAS:** Uses SystemOut.log and SystemErr.log. Look for message IDs starting with WSVR (e.g., WSVR0001I: Server server1 open for e-business).
 - **Liberty Profile:** Look for message codes like CWWKE0001I: The server liberty-server has been launched. inside console.log or messages.log.
-

Configuration Signatures

Spring Boot

- **File names:** application.properties or application.yml.
- **Artifacts:** Usually packaged as an executable standalone .jar file containing an embedded server.

Apache Tomcat

- **Directory:** Located in a conf/ folder.
- **Root elements:** server.xml will feature <Server>, <Service>, <Engine>, and <Connector> XML tags.

WildFly

- **Directory:** Located in standalone/configuration/ or domain/configuration/.
- **Root elements:** standalone.xml features <server xmlns="urn:jboss:domain:..."> and extensive <subsystem> tags.

Oracle WebLogic

- **Directory:** Located within a WebLogic domain directory, e.g., /config/config.xml.
- **Root elements:** config.xml uses <domain xmlns="http://oracle.com"> and defines <server>, <cluster>, and <app-deployment>.

IBM WebSphere

- **Traditional WAS:** Configurations are hidden deep within a cell/node structure (e.g., cells/CellName/nodes/NodeName/servers/ServerName/server.xml).
 - **Liberty Profile:** Uses a simple, clean server.xml file starting with <server description="new server"> and a <featureManager> block listing IBM features.
-

To help me tailor this to your environment, could you tell me:

- Are you trying to automate this detection via a **script** (like Bash or Python)?
- Do you have a **snippet of a log file** or a **directory listing** you want me to look at right now?
- What **operating system** are these applications running on?

AI can make mistakes, so double-check responses

=====

Diagnose HikariCP / DBCP / c3p0 connection pool exhaustion

Connection pool exhaustion in Spring Boot happens when all available database connections are in use, and new requests must wait or fail with a timeout error.

Common Causes

- **Slow queries:** Queries take too long and keep connections busy.
 - **Connection leaks:** Code borrows a connection but fails to close it.
 - **High concurrency:** Traffic exceeds the maximum pool size limit.
 - **Under-sized pool:** The maximum pool size is too small for the load.
-

How to Diagnose

1. Enable Pool-Specific Logging

Add debug or trace logging in your application.properties or application.yml to track connection usage.

- **HikariCP (Default in Spring Boot):**
properties

```
://level.com.zaxxer.hikari=DEBUG
://level.com.zaxxer.hikari.HikariConfig=DEBUG
```

Use code with caution.

HikariCP logs warnings automatically when connections are acquired slowly or when the pool is exhausted. Look for logs containing Apparent connection leak detected.

- **Apache DBCP2:**
properties

```
logging.level.org.apache.commons.dbcp2=DEBUG
Use code with caution.
```

- **C3P0:**

properties

```
://level.com.mchange.v2.c3p0=DEBUG
Use code with caution.
```

2. Monitor Active Metrics with Actuator

Add Spring Boot Actuator to your project to inspect real-time pool metrics.

1. Add the dependency:

xml

```
<!-- Maven -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
Use code with caution.
```

2. Expose the endpoints in application.properties:

properties

```
management.endpoints.web.exposure.include=health,metrics
Use code with caution.
```

3. Visit /actuator/metrics/hikaricp.connections.active or /actuator/metrics/data.source.connections.active to see how many connections are active, idle, or waiting.

3. Check for Connection Leaks in Code

- **HikariCP Leak Detection:** Configure Hikari to log stack traces when a connection is held longer than a threshold (e.g., 2 seconds):

properties

```
spring.datasource.hikari.leak-detection-threshold=2000
Use code with caution.
```

- **Review code paths:** Ensure every manual Connection, Statement, or ResultSet is closed properly, preferably using Java try-with-resources blocks or letting JPA/Hibernate manage transactions correctly.

4. Adjust Pool Sizes

If your application is healthy but under heavy load, increase the maximum pool size cautiously (ensure your database server can handle the increased maximum connections).

- **HikariCP:**
properties

```
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.connection-timeout=30000
Use code with caution.
```

If you'd like, let me know:

- Which **connection pool** you are using (HikariCP, DBCP, or c3p0)
- The **error message** or stack trace from your logs

I can help you pinpoint the exact configuration fix or code change you need.

AI can make mistakes, so double-check responses

=====

•Resolve Spring Boot startup failures (port in use, bean wiring errors, missing properties)

When a **Spring Boot** application fails to start, the embedded **FailureAnalyzer** usually prints a clear, human-readable error report right before the stack trace

Here is how to quickly resolve the three most common startup failures: **Port already in use**, **Bean wiring errors**, and **Missing properties**.

1. Port Already in Use

The Error: Web server failed to start. Port 8080 was already in use.

This happens when another application, database, or a stuck zombie instance of your own application is already listening on the configured port

Solutions:

- **Option A: Free up the port**
Find the process ID (PID) occupying the port and terminate it.
- *Windows:* Open CMD as Administrator and run:

cmd

```
netstat -ano | findstr :8080
taskkill /F /PID <PID_NUMBER>
\\`
```

Use code with caution.

- *macOS / Linux:* Open the terminal and run:

bash

```
sudo lsof -i tcp:8080
kill -9 <PID_NUMBER>
\\`
```

Option B: Change the application port

Assign a new port in your config file:

- In src/main/resources/application.properties:

properties

```
server.port=8081  
\\
```

Use code with caution.

- In application.yml:

yaml

```
server:  
  port: 8081
```

2. Bean Wiring & Dependency Injection Errors

The Error: Consider defining a bean of type 'X' in your configuration. or No qualifying bean of type available.

This occurs when Spring's IoC container cannot locate or uniquely identify a component it needs to inject. [[1](#), [2](#), [3](#)]

Scenario A: Bean Not Found (Missing Component)

Spring doesn't know the class exists.

- **Fix:** Ensure the target class is annotated with a stereotype annotation like @Component, @Service, @Repository, or @Controller.
- **Fix:** Check your packages. By default, Spring Boot only scans classes inside or beneath the package containing the main @SpringBootApplication class. Move your class inside that package structure or explicitly add @ComponentScan("com.yourcompany")

Scenario B: Non-Unique Bean (Ambiguous Injection)

You have multiple classes implementing the same interface, and Spring doesn't know which one to pick. [[1](#)]

Fix using @Primary: Mark your preferred default implementation with @Primary.

- **Fix using @Qualifier:** Give your beans names and use @Qualifier("beanName") alongside @Autowired or your constructor injection to specify the exact bean required

Scenario C: Circular Dependencies

Bean Currently In Creation Exception. ClassA requires ClassB, and ClassB requires ClassA

<https://medium.com/@charansai.ashokit/spring-boot-common-errors-solutions-b1ac63edecc9>

Fix: Refactor your code to break the loop (usually by introducing a third bean or using lazy initialization with @Lazy, though architectural refactoring is preferred)

3. Missing or Unresolved Properties

The Error: IllegalArgumentException: Could not resolve placeholder 'my.property' in value "\${my.property}".

This happens when your code requests an environment or configuration property that hasn't been defined anywhere

<https://medium.com/@puneetpm/demystifying-spring-boot-startup-failures-your-ultimate-troubleshooting-guide-550a9050ef95>

Solutions:

- **Option A: Provide a Safe Default Value**

If the property is optional, supply a fallback value directly inside the @Value annotation using a colon (:):

java

```
@Value("${my.custom.timeout:5000}") // Defaults to 5000 if my.custom.timeout  
is missing  
private int timeout;  
Use code with caution.
```

- **Option B: Define the Value Across Profiles**

Ensure the property is declared in your active configuration file:

properties

```
# application.properties  
my.custom.timeout=5000
```

If you rely on environment-specific files (like application-prod.properties), verify that you actually activated that profile using ``spring.profiles.active=prod``

Option C: Use Strict Validation with @ConfigurationProperties

If the property is critical, group them using @ConfigurationProperties. You can combine this with Hibernate Validator's @NotNull to fail quickly and transparently with a clear validation report, preventing silent configuration issues down the road

=====

- **Use Spring Actuator /threaddump and /heapdump for live diagnosis**

Spring Boot Actuator provides /threaddump and /heapdump endpoints for real-time JVM diagnostics and troubleshooting

Using /threaddump

- **Purpose:** Returns a snapshot of all active threads in the Java Virtual Machine (JVM).
- **Output Format:** JSON by default, or plain text if you set the Accept : text/plain HTTP header.

- **Use Case:** Diagnose deadlocks, high CPU usage, or stuck/hung threads by comparing two dumps taken a few seconds apart.

- **Command Example:**

bash

```
curl http://localhost:8080/actuator/threaddump
```

Use code with caution.

[[1](#), [2](#)]

<https://medium.com/@sarveshkhhamkar321/spring-boot-actuator-your-applications-built-in-health-monitor-7b9692890ef4>

Using `/heapdump`

- **Purpose:** Downloads a complete binary snapshot (`.hprof` for HotSpot JVM) of all objects currently in JVM memory.
- **Output Format:** Large binary stream.
- **Use Case:** Analyze memory consumption, find object accumulation, and troubleshoot memory leaks using tools like Eclipse MAT or VisualVM.
- **Command Example:**

bash

```
curl http://localhost:8080/actuator/heapdump -O
```

Use code with caution.

(Note: Triggering a heap dump pauses the JVM briefly; use it as a break-glass diagnostic tool rather than on a regular polling schedule.) [[1](#), [2](#), [3](#), [4](#)]

Security Best Practices

- **Exposure:** These endpoints are sensitive and disabled or unexposed by default in many configurations. Never expose them publicly without authorization.
- **Protection:** Secure them using [Spring Security](#) or isolate them on a separate internal management port. [[1](#), [2](#), [3](#), [4](#), [5](#)]

=====.

•Resolve Tomcat / WildFly app deployment failures

To resolve Tomcat or WildFly deployment failures when migrating or running a Spring Boot application on an external application server, adjust your build configuration to prevent server and logging conflicts. [[1](#), [2](#)]

Common Fixes for WildFly / External Server Failures

- **Change Packaging to WAR:** Modify your project configuration to produce a WAR file instead of an executable JAR.

- **Extend `SpringBootServletInitializer`:** Update your main application class to extend `SpringBootServletInitializer` so the external servlet container can initialize it.
- **Mark Embedded Tomcat as Provided:** Exclude or scope the embedded Tomcat dependencies as `provided` so they do not conflict with the application server's runtime.
- **Exclude Logback Classic:** Remove `logback-classic` from your dependencies to prevent `LoggerFactory is not a Logback LoggerContext` runtime exceptions inside WildFly.
- **Remove Conflicting Undertow Starters:** Ensure `spring-boot-starter-undertow` is omitted or provided-scoped, since WildFly supplies its own Undertow subsystem.
- **Add Missing Servlet APIs explicitly:** For Spring Boot 3.x, add the `jakarta.servlet-api` dependency with a `provided` scope if servlet initialization fails

=====

•**Configure correct JVM args for containerised Java workloads (heap, GC, container-aware flags)**

Configure JVM arguments for containerized Java workloads by enabling container awareness and setting explicit memory limits to prevent out-of-memory crashes.

Key Container-Aware Flags

Modern Java versions (Java 10+) automatically recognize container memory and CPU limits, but older versions or specific setups need explicit flags.

- `-XX:+UseContainerSupport`: Enables container awareness so the JVM reads limits from cgroups (enabled by default in Java 10+).
- `-XX:MaxRAMPercentage=75.0`: Sets the maximum heap and non-heap memory the JVM can use as a percentage of the container's total memory limit (leave room for off-heap memory like metaspace and thread stacks).
- `-XX:InitialRAMPercentage=50.0`: Sets the initial memory allocation percentage.
- `-XX:MinRAMPercentage=50.0`: Sets the minimum memory allocation for small containers.

Garbage Collector (GC) Selection

Choose a garbage collector based on your latency and throughput needs.

- **G1GC (Default for Java 9+)**: Best for balanced throughput and low latency. Add `-XX:+UseG1GC`.
- **ZGC (Low Latency)**: Best for massive heaps with ultra-low pause times (Java 15+). Add `-XX:+UseZGC`.
- **ParallelGC (High Throughput)**: Best for batch jobs where pause time does not matter. Add `-XX:+UseParallelGC`.

Recommended Example Configuration

Use this production-ready baseline for a modern Java container (Java 17+):

bash

```
java -XX:+UseContainerSupport \
      -XX:MaxRAMPercentage=75.0 \
      -XX:+UseG1GC \
      -XX:+ExitOnOutOfMemoryError \
      -jar app.jar
```

Use code with caution.

- `-XX:+ExitOnOutOfMemoryError`: Forces the container to crash immediately on a `java.lang.OutOfMemoryError`, allowing the orchestrator (like Kubernetes) to restart a clean instance.

If you want, let me know:

- Which **Java version** you are using
- Your container **memory limit** (e.g., 512MB, 2GB)
- Your primary goal (**low latency** or **high throughput**)

I can refine these JVM arguments for your exact setup.

AI can make mistakes, so double-check responses